Technical Design Record: SunGrid AI Copilot

Lead, AI & Data Science case study, Part B. Date 2026-10-28. Status: working prototype (Part A), production design proposed. Diagram: `architecture-diagram.png` (source `architecture-diagram.svg`). How to run everything: repository `README.md`.

What exists. A pipeline of classify, then one of three paths: refuse (non-relevant), retrieve and answer with citations (knowledge), or check Rooftop Rebate eligibility through `stub_tools.check_rebate_eligibility` (unchanged). **What does not.** An HTTP service, a Qdrant server, dashboards and the review queue. They are in the diagram, drawn dashed, and are a design, not a deployment.

1. The three most consequential decisions

D1. A fixed workflow with the eligibility gate in code, not an LLM that chooses tools. Chosen: the classifier sets the intent and code routes on it. For eligibility the model only reads the four inputs out of the message (null when not stated). Code validates them, asks the member for anything missing without calling the service, calls the service with a 5 s timeout, and builds the reply from a template. A dollar figure can only be the service's own estimate when it said eligible, and any error ends in a reply with no figure. Rejected: (a) an LLM function-calling agent. It decides when to call the tool and writes the reply, so a gate is only as strong as a prompt, and it can soften a "no" into an estimate. (b) Regex-only extraction, which breaks on "$85k" or "six kilowatts". Why: the hard gate is a safety property, so I wanted it provable by test (13 tests, no API calls) rather than hoped for. The cost is flexibility: a new tool is a new code path, and the only dialogue is asking for missing fields.

D2. Category as a soft retrieval filter with an unfiltered fallback; document labels assigned by hand and multi-label. Chosen: each document has one or more categories in a registry file. The classifier's primary label (and its secondary label unless confidence is high) filters the Qdrant search. If the best chunk anywhere beats the best in-filter chunk by more than 0.05 cosine, the label is treated as wrong and the search repeats unfiltered. The ambiguous document (06, rebate refund / billing adjustment) carries both `incentive_rebate` and `billing_account`, so it is reachable under either label. Rejected: (a) a hard filter, where a wrong label makes the right chunk unreachable with no error; (b) no filter, which works at 69 chunks but gives up the lever needed at 10x; (c) routing the pipeline per category, which the brief rules out and which fails on document 06; (d) LLM-assigned document labels, which are costly and not reproducible at scale, and for 14 documents a human is cheaper and auditable. Evidence, stated plainly: at this corpus size the filter gave no measurable gain, so recall is the same with or without it. What the fallback buys is that the filter cannot hurt. The 0.05 margin was chosen while looking at these same questions, so it is optimistic.

D3. Grounding enforced in code, not by a similarity threshold or a prompt alone. Chosen: the answer step returns `answerable`, the answer and the chunk ids it cites. Code rejects any citation that is not among the chunks it was shown and replies "not found" instead. Chunks from doc 05's unreleased section are labelled in the prompt as not current policy. Rejected: (a) a score threshold for abstaining: answerable questions scored 0.74 and above but unanswerable probes reached 0.70, a gap too narrow to trust; (b) a prompt saying "say you don't know", with no check. What it caught: the first full run produced false "not found" replies because the model wrote ids in brackets. I fixed that (strip brackets, still require membership in the retrieved set). Limit: a valid citation does not prove the claim is supported. That is only checked offline by the judge, not at runtime.

2. Evaluation

Offline (Part A, all from `eval/reports/`; 20 questions plus probes, 3 runs each, one author, so weak evidence, not an accuracy estimate).

Check	Result
Retrieval, 14 questions with gold chunks, no filter	recall@1 0.86, recall@5 1.00, MRR 0.90
Classifier, 20 x 3	category 60/60, intent 60/60
Eligibility path, 8 cases x 3	routed 24/24, inputs exact 24/24, gate respected 24/24
End to end, 23 questions x 3, Sonnet 5.5 judge	66/69 (96%): knowledge 33/36, ambiguous 6/6, eligibility 12/12, non-relevant 6/6, unanswerable probes 9/9

The 3 failures are one question, q03 (voting weight, docs 01 and 08 disagree). The answer comes from doc 08 alone, because doc 01's relevant chunk ranks 9th and the model sees only 5. The citation fix above came after seeing results (63/69 before, 66/69 after), so 96% is not a clean held-out figure. recall@5 is saturated because 5 chunks are 7% of the corpus. The LLM judge was spot-checked on only 4 answers.

Monitoring in production. Every request already writes a trace (classification, chunks and scores, tool inputs and outputs, answer, cost, latency). Proposed: - *Dashboard and alerts.* Rate of abstentions, fallbacks, rejected citations, and the split of eligibility outcomes (eligible, ineligible, needs info, error). p95 latency and cost per query. **Page immediately** on any dollar figure in a non-eligible reply (must be zero). Ticket when the abstain or fallback rate doubles against its four-week average (starting thresholds, to be tuned). - *Human labels.* Staff review a 5% random sample plus every abstention, rejected citation and thumbs-down. Their labels are added to the eval set and used to check the LLM judge, which is unvalidated today. -

Change control. CI reruns the eval on any change to a prompt, a document, the registry or the model ID. The model ID is pinned, and moves only after an eval run.

3. Scale

10x documents (about 140 documents, about 700 chunks). The vector store is not what breaks first. 1. *Retrieval recall and conflicts.* q03 already shows that top-5 can miss the second half of a disagreement at 14 documents. With more near-duplicates and overlapping policies this gets worse. Change: retrieve about 20, rerank to 5, add a conflict check over chunks about the same entity, and carry owner and effective date on every document. 2. *The hand-labelled registry and full re-ingest.* Change: an LLM proposes labels and a person approves them, and ingestion re-embeds only documents whose content hash changed. 3. *Qdrant local mode* locks its folder to one process and warns above about 20k points. Fine at 700, but it forces the move to a server (below).

100x queries. 1. *Single process.* Local mode and the CLI cannot serve concurrent replicas. Change: Qdrant server, stateless API replicas behind a load balancer. 2. *Anthropic rate limits and tail latency* (median 3.1 s, max 5.1 s in my runs). Change: concurrency limits, retries with backoff, and a fixed "contact support" reply when the provider is down. Cache answers to repeated knowledge questions, keyed by normalised question plus knowledge-base version. Never cache eligibility replies. 3. Cost grows linearly (section 4).

4. Cost

Item	Estimate
Runtime, Haiku 5.5 ($0.10 / $0.50 per million input / output tokens)	**about $0.40 per 1,000 queries**, measured from token usage across the eval runs (classifier alone $0.19 per 1,000 calls)
Same workload entirely on Sonnet 5.5 ($2 / $10 per million)	about $8 per 1,000, from the price ratio (20x), not run
Embeddings	$0 per query: bge-small runs on the API's own CPU
Eval judge (Sonnet 5.5)	about $0.09 per full run, offline, not a runtime cost
Hosting (API replicas, Qdrant server, storage, logs)	not measured, and at low volume it will dominate the model bill

Keeping it predictable: forecast as queries x measured cost per query and put the cost in every trace; cap output tokens and keep retrieval at a fixed 5 chunks, so input size is bounded; per-member rate limits; a daily spend alarm with a hard cap that falls back to a "contact support" reply; the repeated-question cache; the pinned model; and no model upgrade without an eval run.

5. Failure modes: the five most likely wrong or harmful answers

#	Failure	Likelihood	Mitigation (B = built, P = planned)	Human review
1	Conflicting documents; the answer follows one and states it as fact (q03, seen 3/3)	**High**, observed	B: prompt tells the model to name conflicts; the conflicts are listed in the README. P: rerank, conflict check, document owners	Content owners resolve conflicts surfaced from sampled traces
2	Wrong eligibility result from misread or wrong inputs, or an injected instruction in the message	Medium	B: model fills explicit values only, code validates, the service decides, the reply is a template, no figure unless eligible, and one injection probe (p3) passes (one probe, not a test of robustness). P: show the member the inputs used so they can correct them; word the figure as an estimate. Cannot be detected: a member who gives complete but wrong details	Staff confirm any rebate before it is paid
3	Unreleased or out-of-date content given as current policy (doc 05 fourth tier)	Medium	B: unreleased chunks labelled in the prompt; one eval question covers it and passes. P: effective dates and expiry review on every document	Content owners review expiry
4	Unsupported claim that carries a valid-looking citation	Low to medium	B: citations must be retrieved chunks; judge found no contradictions in 69 runs (judge not validated). P: judge on sampled production traffic	5% sample plus all thumbs-down
5	Over-confident answer where a person must decide (refund reversals and clawbacks, billing disputes: the ambiguous document 06), or a wrongly refused in-scope question (the EV-charger financing question was labelled non-relevant)	Medium	B: prompt forbids personal eligibility verdicts; doc 06 reachable under both labels. P: hand off these intents to staff with a ticket	Staff own refunds, reversals and disputes

6. Change: a new document category next month

Changes (small and local): one registry entry per new document, using the new value; the new value added to the category lists in the classifier and registry validation, plus a definition and examples in the classifier prompt; re-run ingestion; add at least three questions for it to the eval set and re-run the classifier eval, which doubles as a regression check on the existing 20. Review existing labels in case the new category overlaps them, as document 06 does. **Stays the same:** chunking, embeddings, the Qdrant collection and filter, the fallback, the answer step and its citation check, the eligibility component and gate, tracing. **One exception:** if the new category needs its own service call, that is a new intent and a new component built like eligibility, not a change to retrieval.

7. Scope: what I cut, and one non-obvious trade-off

Cut, given 14 to 18 hours: no web UI, login or multi-turn memory; no reranker (BM25 fusion is built but off by default, and was not evaluated as an improvement); no server deployment or infrastructure code, so the diagram is a design; no redaction of member text in traces; no held-out classifier set (the classifier prompt was written after I read document 06, so 60/60 is optimistic); no feedback button. I spent the time on the gate and the evaluation because the brief weights both, and a measured modest score is worth more than an unmeasured high one.

Trade-off: I left q03 failing instead of raising k. Raising k from 5 to 10 probably fixes it and would turn 96% into a higher number. But k applies to every answer, so it adds cost, noise and more chances to cite the wrong chunk for all of them, to fix one question out of 20 that I had already seen. The real fix is rerank plus conflict detection, which is in section 3. I report the failure so that nobody reads the score as better than it is.